

NAME

kh – Kepler's Horizon game server: a two-player tactical game of interstellar combat, fleet command, and economic conquest

SYNOPSIS

kh [*OPTIONS*]

kh **--help**

kh **--version**

kh **--port** *PORT* **--dbusr** *USER* **--dbpass** *PASS* **--dbname** *DB* [**--dbhost** *HOST*]

kh **--ai** *PATH* [**--log** *FILE*] [**--monitor**]

DESCRIPTION

Kepler's Horizon is a deterministic, two-player tactical game of interstellar warfare. Players build individualized warships, navigate a hex-based star map connected by warplines, and resolve combat through strategic power allocation—without dice. The objective is to occupy enemy base star hexes and accumulate victory points.

The game supports single-player mode against an autonomous AI opponent implemented in Common Lisp via the Embedded Common Lisp (ECL) runtime. The AI plays by the same rules as human players, issuing the same commands through the same game engine.

This manual is the authoritative reference for the server, its command language, the AI agent architecture, the economic system, and the Milieu Codex of Stars.

A note on design philosophy. Kepler's Horizon is intentionally minimal. Strategic depth emerges from constrained movement, simultaneous decision-making in combat, and irreversible consequences. Equal inputs always yield equal outcomes. The rules favor clarity over complexity.

Syntax of the Manual

The game accepts text input from the user. The game is designed to treat text case-insensitive. The UI enforces this by making everything look upper case. Throughout this manual page where lower or mixed case appears, it is assured that the game itself is case-insensitive. The grammar is more important for command invocations. Case is irrelevant.

A Note About the Origin and Development

The game idea was a mixture of my own game developed back in 1984 and influenced heavily by other space-war games, such as WarpWar (1977). The implementation of the software of this game was designed and architected by a human, but some features were initially implemented by AI (Opus 4.6 via Anthropic/Claude).

I make no qualms about using AI to develop parts of the software, as this was helpful in speeding up the process. However, all of the software had to be revised anyway. Further, the core engine of the parser (**lex** and **yacc**) was hand-written.

The *LISP* code used by the AI-Agency was mostly authored with AI support, but the ontology was designed and implemented by human work. Again, I have no qualms about this because it is effectively the same as spending days researching. The reader may disagree.

After 45 years of writing software, the author can attest that fundamental breakthroughs in some kinds of software projects are greatly aided by AI-assisted engineering. There is no looking back.

OPTIONS

Generic Program Information

--help Print a usage summary and exit.

-v, --version

Print the version number and git SHA, then exit.

Database Connection

-H, --dbhost HOST

MySQL server hostname. Default is **127.0.0.1**.

-u, --dbusr USER

MySQL user name.

-d, --dbname DB

MySQL database name (typically **khdb**).

-p, --dbpass PASS

MySQL password.

Network

-P, --port PORT

HTTP port for the REST API. Default is **8080**.

AI Configuration

--ai PATH

Path to the directory containing AI DSL files (Common Lisp sources). Default is **dsl** relative to the working directory. The server loads eight Lisp files from this directory on first single-player game start: *aa-util.lisp*, *aa-strategy.lisp*, *aa-goals.lisp*, *aa-economic.lisp*, *aa-build.lisp*, *aa-movement.lisp*, *aa-combat.lisp*, *aa-core.lisp*.

Logging

-L, --log FILE

Write server log output to *FILE*.

Monitor Mode

-M, --monitor

Enable terminal command input on the server console.

GETTING STARTED

Web Portal

Login or create an account at the web portal. The portal serves static HTML pages and communicates with the game server through a REST API.

Lobby

The lobby displays open game rooms. A player can create a new room (which selects a universe module) or join an existing room by taking seat A or seat B.

Starting a Game

When both seats are filled, either player may start the game. In single-player mode, an AI user occupies the second seat automatically.

Module Selection

The game host selects the universe module at room creation time. A **Module** defines a complete game universe: map topology, star systems, planets, species, resources, anomalies, and facilities. The game engine is module-agnostic; only content varies between modules.

The default module is **Kepler's Horizon**: 28 star systems across Alpha and Beta territories, with a Contested Reach between them.

MAP COMPONENTS

The game map is a hex grid overlaid with named star systems and warpline connections. Not every hex contains a star.

Star Hex

A map hex containing a star system (large or small). Star hexes are the nodes of the warpline graph. Ships must stop at enemy-occupied star hexes.

Base Star Hex

A star hex containing a large star. There are six base stars total, three per side (Alpha and Beta). Occupying enemy base stars earns victory points.

Space Hex

A hex with no star and no warpline. Ships may pass through freely.

Warp Line

A bi-directional edge connecting two star hexes. Traversing a warpline costs 1 PD regardless of distance. Warplines pass through intermediate hexes but those hexes are treated as space hexes for movement purposes.

SHIP TYPES AND ATTRIBUTES**Ship Types****WarpShip (W1–W9)**

Capital ships capable of interstellar travel via warplines. WarpShips carry a Warp Generator (5 BP, mandatory) and may equip SystemShip Racks to carry escort vessels. Hull codes are W followed by a single digit: W1, W2, ... W9. A fleet may contain up to 9 WarpShips.

SystemShip (S01–S99)

Escort vessels that operate within a single star hex. SystemShips have no Warp Generator and cannot traverse warplines. They must be carried by WarpShips in SystemShip Racks. Hull codes are S followed by one or two digits: S1, S11, S25, etc.

Ship Attributes

Every ship has the following combat-relevant attributes, each costing Build Points (BP) at construction time:

Attribute	Code	Cost	Description
Power/Drive	PD	1 BP/unit	Engine strength; powers movement and combat
Warp Generator	WG	5 BP	WarpShips only (automatic, mandatory)
Beam	B	1 BP/unit	Energy weapon; damage = allocation + tech level
Screen	S	1 BP/unit	Energy shield; absorbs = allocation + tech level
Tube	T	1 BP/tube	Launches 1 missile per combat round
Missiles	M	1 BP/3	Projectile weapons; damage = 2 + tech level
SystemShip Racks	SR	1 BP/rack	Carries 1 SystemShip per rack (WarpShips only)

Additionally, ships may be equipped with economic hardware:

Equipment	Code	Cost	Purpose
Long Range Scanner	LRS	50 CR	Required for extract scan and survey
Transporter Beam	TB	75 CR	Enables resource extraction
Mining/Salvage Drones	DR	100 CR	Enables extraction and salvage operations

Equipment does not affect combat. A ship is destroyed only when all combat attributes (PD, B, S, T, M, SR) are reduced to zero. The Warp Generator never takes damage.

Technology Levels

Ships retain their original tech level permanently. Tech level is determined by the game round at construction time:

Rounds	Tech Level
1–4	Level 0
5–8	Level 1
9–12	Level 2
13–16	Level 3
17+	+1 per 4 rounds

Tech level adds to:

- Beam damage (when hitting)

- Missile damage (base 2 + tech level)
- Screen absorption (when powered)

A Tech Level 2 ship with Beam 3 allocated to beams deals 5 damage on a hit. A Tech Level 2 ship with Screen 2 allocated absorbs 4 hits.

GAME SEQUENCE

Each player-turn consists of the following phases, in order. The command **next** (alias **n**) advances to the next phase. The command **done** (alias **ZZ**) ends the current player's turn and passes initiative to the opponent.

1. **Count Victory Points.** At the start of each player-turn, the system awards 1 VP for each enemy base star hex occupied by your ships.
2. **Build Ships.** Receive Stellar Credits (CR) and spend Build Points (BP) to construct new ships, repair damaged ships, resupply missiles, and manage equipment.
3. **Movement.** Move WarpShips along hex paths and warplines. Pick up and drop off SystemShips. Deploy newly built ships to your base star hexes.
4. **Resolve Combat.** At every star hex where opposing ships coexist, combat is triggered. Players issue simultaneous orders, resolve fire, assign damage, and handle retreats.
5. **SystemShip Pick and Drop.** Free rearrangement of SystemShips at star hexes. No PD cost. Active player only.
6. **End of Turn.** Finalize the turn. Market prices update after both players complete a round.

COMMANDS

Commands are case-insensitive. Short aliases are shown after commas. All commands are entered in the console text field on the game page.

Session Commands

save *name*

Save the current game state under *name*. The name must match the lexer pattern **[a-zA-Z][-a-zA-Z0-9]***: letters, digits, and dashes only. No underscores, dots, or spaces. Without arguments, shows usage.

load *name*

Propose loading a previously saved game. This initiates a two-factor confirmation: the other player must type **accept** *name* to confirm. Without arguments, lists available saves.

accept *name*

Accept a pending load request from the other player. The game state is restored from the saved snapshot.

reject *name*

Reject a pending load request.

delete *name*

Delete a saved game permanently.

quit

Exit the current game. The game is auto-saved as **autosaved-YYYY-MM-DD-HH-MM**. Combat state is nullified (ships remain in position; combat will re-trigger on reload). The other player receives a notification and is redirected to the lobby. The AI thread's game context is zeroed so it naturally ceases operating on this game.

clear, cls

Clear the console display.

Turn Management

next, n Advance to the next game phase. Blocked if there are pending retreats or unresolved damage.

done, ZZ

Complete the current player's turn and pass initiative to the opponent. Blocked if there are pending retreats.

Information Commands

score Display victory points and credits for both players, plus current round and tech level and display current game state: whose turn, phase, round, credits, tech level.

fleet Display a manifest of your entire fleet: every ship with its hull code, name, location, and current attribute values (PD, B, S, T, M, SR, equipment).

cargo ship

Display the cargo manifest for *ship*: quantities of each resource type in the hold, cargo capacity, and missile storage.

hex location

Show information about hex *location*: which ships occupy it, which star system (if any), active hex events, and warpline connections.

system name, sy name

Show information about star system *name*. Subcommands:

system name anomalies

List anomalies at the system.

system name facilities

List facilities at the system.

system name resources

List extractable resources at the system.

system name planets

List planets and moons in the system.

survey name, sv name

Survey a star system to increase your knowledge level (see MILIEU CODEX). Requires a ship with LRS at the target system. Each survey operation upgrades knowledge by one level. Without arguments, shows survey status.

crt Display the Combat Results Table.

help topic, ? topic

Show help for a command or topic. Without arguments, lists all available topics. Topics: general, build, move, combat, economy, info, turn, ship, crt, missiles.

recap Display a summary of recent game events.

Build Commands**build, bb**

Show current build state: active drafts and available BP.

build new class name, bn class name

Create a new ship draft. *class* is W (WarShip) or S (SystemShip), optionally followed by 1–2 digits for a specific hull number (e.g., W4, S11). If no number is given, the game auto-assigns the next available hull code. *name* is the ship designation (e.g., Falcon, Avenger).

Examples:

> BN W FALCON

> BN S11 AVENGER

> BUILD NEW W4 DESTROYER

build drafts, bd

List all pending build drafts.

build drafts *ship*, **bd** *ship*

Show detailed attributes of a specific draft (by code or name).

build set *target attributes*, **bs** *target attributes*

Set attributes on a draft ship. *target* is ship code or name. Attribute syntax:

PD=N or D=N

Power/Drive (1 BP per unit)

B=N Beam weapon (1 BP per unit)

S=N Screen defense (1 BP per unit)

T=N Missile tubes (1 BP per tube)

M=N Missiles (1 BP per 3 missiles)

SR=N SystemShip racks (1 BP per rack, WarpShips only)

Example: BS FALCON PD=7 B=3 S=2 T=1 M=6

build commit *target*, **BC** *target*

Finalize a draft and add the ship to your fleet. BP is deducted. The ship appears in the fleet but must be deployed before it can act.

build cancel *target*, **bx** *target*

Cancel a draft. No BP is deducted.

Deployment Commands**deploy** *ship system*, **ds** *ship system*

Deploy a newly built ship to a base star system.

The first deployment by either player claims a side of the map. The three base stars on that side become your home bases. Subsequent deployments must go to one of your claimed base stars.

Movement Commands**move** *ship dest [via...]*, **m** *ship dest [via...]*

Move a WarpShip to *dest*, optionally through intermediate waypoints. Waypoints may be star system names or hex IDs (e.g., h1310). Movement costs 1 PD per hex or per warpline jump.

Critical rule: WarpShips are forced to stop at any star hex occupied by enemy ships. Combat is triggered at the end of the movement phase.

PD expended for movement is *not* permanently reduced. Full PD is available for combat power allocation.

Example: M W6 TAVON XYLEN

W6 moves to Tavon, then onward to Xylen.

pick *systemship warpsip*, **pu** *systemship warpsip*

Load a SystemShip onto a WarpShip's rack. Costs 1 PD during the movement phase. Ships must be at the same hex.

drop *systemship warpsip*, **do** *systemship warpsip*

Unload a SystemShip from a WarpShip. Costs 1 PD during the movement phase.

Combat Commands**combat**, **cm**

Show active combat status at all contested hexes.

combat order *ship tactic target allocation*, **co ...**

Issue a combat order for one of your ships.

Tactics:

a (attack)

Aggressive offense. Best at 0 to +2 drive advantage.

d (dodge)

Defensive maneuver. Good at +1/+2 drive, hard to hit at 0/-1.

e (escape/retreat)

Attempt to flee combat. WarpShips only. SystemShips may never select escape.

Allocation: pd=N b=N s=N t=N m=N

Beam fire example:

> co W3 a S25 pd=2 b=3 s=2

W3 attacks S25: 2 drive, 3 beam, 2 screen.

Missile fire example:

> co S25 d W3 pd=4 t=1 m=3

S25 dodges while firing 1 missile at W3 with missile drive=3. Each **m=** parameter specifies one missile's drive power.

combat commit, cc

Lock in all pending combat orders. Once both players commit, the combat round resolves automatically.

combat cancel, cx

Cancel all pending combat orders.

combat drafts, cd

Show your pending (uncommitted) combat orders.

combat apply ship allocation, ca ship allocation

Assign damage to a ship's attributes after combat resolution.

Example: ca W3 pd=2 b=1 s=2

Distributes 5 total damage: 2 to PD, 1 to Beam, 2 to Screen. The total assigned must equal the damage dealt (or consume all remaining HP if the ship is destroyed).

retreat ship hex, rt ship hex

Retreat a ship from combat to an adjacent hex. Retreat destinations are validated against the hex adjacency graph.

Repair and Resupply Commands**repair ship attr=N, rp ship attr=N**

Repair a damaged ship. 1 BP per attribute point restored. Ship must start the turn at a friendly base star hex (or at a REPAIR_DOCK or SHIPYARD facility). Cannot repair beyond original built capacity. One attribute per command invocation.

Example: rp W3 pd=2

resupply ship quantity, rs ship quantity

Resupply missiles at 1 BP per 3 missiles (up to original capacity). Ship must be at a friendly base star hex.

Economy Commands**extract scan, ex scan**

Scan the current system for harvestable resources. Requires a ship with LRS (Long Range Scanner) at the system. Yield is modified by your knowledge level (see MILIEU CODEX).

extract ship resource, ex ship resource

Extract a resource from the current system into the ship's cargo hold. Requires TB (Transporter Beam) or DR (Drones). Multi-word resources use spaces: **ex W1 rare earth**.

market, mk

Display current market prices with trend indicators (RISING, STABLE, FALLING).

market resource, mk resource

Show price history for a specific resource.

trade list, tr list

Show base trade prices for all resources.

trade buy *resource quantity*, **tr buy** ...

Purchase resources at the current market price, deducting credits.

trade sell *resource quantity*, **tr sell** ...

Sell resources from cargo at 75% of market price.

trade transfer *from to resource qty*, **tr ts** ...

Move resources between ships at the same location.

fabricate list, fab list

Show available fabrication recipes and their material costs.

fabricate *recipe [quantity]*, **fab** *recipe* ...

Manufacture items from raw materials in cargo.

outfit *ship lrs|tb|drones*, **out** *ship* ...

Install equipment on a ship.

outfit list, out list

Show available equipment and their credit costs.

salvage scan, jk scan

Scan for salvageable objects at the current hex. Requires DR (Drones). Discovery chance is modified by knowledge level.

salvage *ship*, **jk** *ship*

Conduct a salvage operation at the current hex. Requires DR. Hazard chance is modified by knowledge level.

Development Commands**gamedev, gd**

Show current development override status.

gamedev *subcmd value*, **gd** *subcmd* ...

Set development overrides. Subcommands: env, combat, move, crt.

gamedev reset, gd reset

Clear all development overrides.

COMBAT RESOLUTION

Combat is the heart of Kepler's Horizon. It is entirely deterministic: given the same orders, the same results will always occur. There are no random rolls.

Combat Trigger

Combat is triggered when opposing ships occupy the same star hex at the end of the movement phase. The system creates a **combat_state** record for that hex and notifies both players.

Combat Stages

Combat at each hex proceeds through four stages in a cycle:

Stage	Name	Description
0	ORDERS	Both players issue orders for their ships
1	RESOLVE_READY	System resolves the round (automatic)
2	DAMAGE_PENDING	Players assign damage to their ships
3	RETREAT_PENDING	Stalemate: initiative player must withdraw

The Combat Results Table (CRT)

The CRT is the core lookup table that determines whether a weapon strikes its target. It is indexed by three values:

1. Firing ship's tactic (Attack, Dodge, or Retreat)
2. Target ship's tactic

3. Drive Difference = Firing Drive – Target Drive

Firing	Drive Diff	vs Attack	vs Dodge	vs Retreat
Attack	–3 or less	Miss	Miss	Escapes
Attack	–1, –2	Hit	Miss	Escapes
Attack	0, +1	Hit+2	Miss	Miss
Attack	+2	Hit+1	Hit+1	Miss
Attack	+3, +4	Miss	Hit	Hit
Attack	+5+	Miss	Miss	Miss
Dodge	–4 or less	Miss	Miss	Escapes
Dodge	–2, –3	Miss	Hit	Escapes
Dodge	0, –1	Hit	Hit	Escapes
Dodge	+1, +2	Hit	Miss	Escapes
Dodge	+3+	Miss	Miss	Escapes
Retreat	–2 or less	Miss	Miss	Escapes
Retreat	–1, 0	Hit	Miss	Escapes
Retreat	+1+	Miss	Miss	Escapes

Observe:

- Attack vs Attack at 0/+1 drive difference yields Hit+2—the deadliest intersection. Aggressive players who match drives annihilate each other.
- A drive difference of +5 or greater is always a miss—the attacker overshoots. This prevents dump-stat strategies.
- Dodge is safest at 0/–1 vs Attack (miss), but vulnerable at 0/–1 vs Dodge (hit).
- Retreat succeeds ("Escapes") against any tactic at favorable drive differentials, but fails if the enemy's drive advantage is too high.

Power Allocation

Each combat round, a player allocates their ship's PD to subsystems:

Drive (D)

Maneuverability. Determines the row/column on the CRT. Higher drive = better chance to hit or dodge, but diverts power from weapons and shields.

Beam (B)

Weapon power. Cannot exceed the ship's built beam rating. Damage on hit = beam allocation + tech level.

Screen (S)

Defensive shield. Cannot exceed the ship's built screen rating. Absorption = screen allocation + tech level.

Tubes (T)

Number of missiles to fire. Each tube requires 1 PD.

Constraint: $D + B + S + T \leq \text{current PD}$.

Critical rule: Beams/Screens and Missiles cannot be used in the same round. If you fire missiles, you cannot power beams or screens.

Beam Damage

When a beam attack hits:

1. Damage = Beam power allocation + Tech level
2. If target powered screens: absorption = Screen allocation + Tech level
3. Net damage = $\max(0, \text{damage} - \text{absorption})$

Missile Damage

Each missile is individually resolved:

- Missiles always use the **Attack** tactic on the CRT
- Each missile has its own drive setting (specified via **m=N**)
- Damage per hitting missile = 2 + Tech level
- Missiles are expended on firing (regardless of hit/miss)
- Each tube fires at most 1 missile per round

Damage Assignment

After resolution (Stage 2), each player must assign their ship's damage to specific attributes using **combat apply**.

- Total assigned must equal damage dealt
- If damage exceeds remaining HP, the ship is destroyed (overkill accepted)
- Ship is destroyed when $PD + B + S + T + M + SR = 0$
- The Warp Generator is never damaged

Retreat

A ship using the Escape tactic must receive "Escapes" results against *all* enemy ships firing at it. If any attacker rolls a Hit, the retreat fails and the ship remains in combat.

On successful retreat, the ship is flagged **escape_pending**. The player must issue **etreat ship hex** to move the ship to an adjacent hex. The turn is blocked until all retreats are completed.

SystemShips may never retreat. They must be carried out by a WarpShip using the Pick/Drop mechanism.

Stalemate

If three consecutive combat rounds produce zero unabsorbed damage (`stalemate_counter >= 3`), the initiative player (who moved into the hex) must withdraw all their ships from the hex. The defender retains control.

Combat End Conditions

Combat at a hex ends when:

- All ships of one side are destroyed or retreated
- Stalemate forces the initiative player to withdraw

MOVEMENT RULES

1. WarpShips **must stop** at star hexes occupied by enemy ships.
2. WarpShips may move through space hexes with enemy ships.
3. Warplines are treated as space hexes for movement purposes. Entering a warpline at one end and exiting at the other costs 1 PD.
4. Ships with $PD=0$ cannot move.
5. PD used for movement is **not permanently expended**. Full PD is available for combat power allocation. This is a critical rule: movement does not weaken your ships for battle.
6. Cannot move onto enemy base star hex during turn 1.
7. Hex events (NAVIGATION_HAZARD) may increase movement cost through specific hexes.

The server uses BFS pathfinding to calculate minimum PD expenditure. Players may specify intermediate waypoints to force a specific route.

VICTORY CONDITIONS**Dominance Victory**

Accumulate 3 victory points by occupying enemy base stars at the start of your turn. Each enemy base star hex with at least one of your ships earns 1 VP.

Elimination Victory

Destroy all enemy ships.

Draw Neither player has effective ships remaining.

THE ECONOMY

The economic layer adds resource management, trading, and manufacturing to the tactical combat game-play.

Currency: Stellar Credits (CR)

All economic transactions use Stellar Credits. Players earn credits from trade, controlling TRADE_HUB facilities (+5 CR/turn), and selling extracted resources.

Resources

Eight types of raw materials exist in the universe. They are found in planets, moons, and asteroid belts, with varying abundance and extraction difficulty.

Resource	Abbrev	Base Price	Primary Uses
FERROUS	Fe	5 CR	Hull construction, fabrication
RARE_EARTH	Re	20 CR	Electronics, screens, tech
RADIOACTIVE	Rd	30 CR	Power, weapons, fabrication
CRYSTALLINE	Cr	25 CR	Beams, warp drives, tech
VOLATILE	Vo	8 CR	Fuel, missiles, fabrication
WATER	H2O	3 CR	Life support
ORGANIC	Or	6 CR	Food, medicine
EXOTIC	Ex	100 CR	Special technology, advanced fabrication

Resource Abundance

Each deposit has an abundance rating that determines base yield:

Abundance	Base Yield
Rich	16 units
High	8 units
Moderate	4 units
Low	2 units
Trace	1 unit

Extraction difficulty further modifies yield:

Difficulty	Modifier
Easy	100%
Moderate	70%
Difficult	40%
Extreme	20%

Knowledge level at the system also modifies extraction yield (see MILIEU CODEX section).

Market Dynamics

The commodity market fluctuates based on supply and demand:

- Prices update each round after both players complete turns
- Buying increases demand, pushing prices up
- Selling increases supply, pushing prices down
- Prices are clamped to 50%–200% of base value
- Price trends are displayed as RISING, STABLE, or FALLING

Fabrication Recipes

Raw materials can be converted into ship upgrades and munitions:

Recipe	Output	Time	Material Cost
--------	--------	------	---------------

missiles	4 missiles	1 round	2 Fe, 1 Rd, 1 Vo
tubes	+1 tube	3 rounds	5 Fe, 3 Re, 2 Cr
beams	+1 beam	3 rounds	8 Fe, 4 Re, 3 Cr
screens	+1 screen	3 rounds	6 Fe, 2 Re, 4 Cr
tech	+1 tech level	5 rounds	10 Re, 5 Cr, 2 Ex
drone	1 scout drone	2 rounds	3 Fe, 2 Re, 1 Ex

Salvage

Wreckage and debris at hex locations can be salvaged for resources. Salvage requires Drones (DR) equipped on the ship.

Discovery: Scanning (**salvage scan**) finds salvageable sites. Discovery chance is modified by knowledge level at the system. Discovered sites are remembered permanently.

Hazards: Each salvage operation carries a hazard chance (typically 5–25%). Hazard types include unstable reactors, booby traps, structural collapse, and hostile salvagers. Hazard damage is dealt to PD. If PD reaches 0, the ship is destroyed. Knowledge level reduces hazard chance.

Yields: Military wrecks yield missiles and rare electronics. Commercial wrecks yield common resources and credits. Ancient derelicts yield exotic materials. Debris fields yield ferrous and basic materials. Sites deplete after multiple operations.

Facilities

Facilities at star systems provide strategic advantages to the controlling player. Control is established by occupying a system with no enemy ships for 2 consecutive turns.

Facility	Effect
SHIPYARD	Build new ships, fabrication base
REPAIR_DOCK	Repair damaged ships away from home base
REFINERY	Fabrication bonuses
TRADE_HUB	+5 CR income per turn, trade access
FORTRESS	Combat defensive bonus

THE MILIEU CODEx

The **Milieu Codex of Stars** is the compendium of known space. It contains detailed information about each star system in the game universe: celestial bodies, planetary environments, native species, resources, anomalies, and strategic notes.

The Codex is not merely flavor text. **Knowledge level directly affects gameplay outcomes.** A commander who surveys a system before exploiting it will extract more resources, discover more salvage sites, and suffer fewer hazards than one who operates blind.

Knowledge Levels

Each player tracks knowledge independently per system. Knowledge starts at **Unknown** and is improved by surveying.

Level	Description	How Gained
Unknown	Name only; no actionable intelligence	Default
Chartered	Basic characteristics; partial star data	First visit
Surveyed	Detailed analysis; full resource data	survey command
Intimate	Complete secrets; hidden sites revealed	survey (second pass)

Gameplay Impact of Knowledge

Extraction Yield Modifiers:

Level	Yield	Explanation
Unknown	25%	Prospectors operate blind
Chartered	50%	Basic geological data improves targeting
Surveyed	100%	Precision extraction with detailed surveys
Intimate	100%	No additional bonus beyond Surveyed

Salvage Discovery Modifiers:

Level	Discovery Chance	Explanation
Unknown	50% of base	Searching blind
Charted	75% of base	General debris fields mapped
Surveyed	100% of base	Precise wreck catalogs available
Intimate	125% of base	Insider knowledge of concealed salvage

Salvage Hazard Modifiers:

Level	Hazard Modifier	Explanation
Unknown	+20%	Operating blind in debris fields
Charted	+10%	Partial hazard mapping
Surveyed	+0%	Standard risk assessment
Intimate	-10%	Know which wrecks are dangerous

Knowledge is **faction-wide**: all your ships benefit immediately when knowledge is gained at a system.

Territories

Systems are organized into three territories:

Western Alliance (Alpha Faction) — Home territory of the Terran alliance.

System	Hex	Star	Key Features
ARVEN	0307	G2V Yellow	Capital. Garden world Verdance. The Anvil shipyards.
BELIX	0606	Binary K1V+M4V	Ancient system. Haven cultural center. Whisper Gate anomaly.
CAYRU	0804	F8V Yellow-White	Young star. The Shatter belt, rich in metals.

Eastern Powers (Beta Faction) — Territory of the Zarekites and their allies.

System	Hex	Star	Key Features
ZAREK	2125	A3V White	Zarekite homeworld. Servitor shipworks. The First Library.
ASTREX	2223	G5V Yellow	Hanging Gardens. Commerce trade zone. Ishtar Gate anomaly.
BRION	2622	K5V Orange	Ancient depleted star. Dagan's Rest breadbasket.

Contested Reach — Disputed systems between the powers.

System	Hex	Star	Key Features
KORAL	1310	G8V	The Axis. 4 warline connections. Axis Marker beacon.
QUELL	1616	M2V Red	Lithoid xenoforms. Keplerite exotic warp-crystals.
SYDRA	1719	K2V Orange	War-torn. The Graveyard and Scrapyrd salvage sites.
VARYN	1922	G1V Yellow	The Remnant machine civilization.
WEXAR	2020	F7V	Kethrani aquatic traders. Water worlds.
XYLEN	2118	F5V	Nexus free port. Mercenary strongholds.
TAVON	1817	M1V Red	Vesh hive mind. The Song anomaly.

Xenoform Species

Species	Homeworld	Traits
Terran Human	Multiple	Adaptable, ambitious colonizers
Zarekite	ZAREK	Light-adapted, hierarchical, ancient tech
Lithoid	QUELL	Silicon-based, expert miners
Kethrani	WEXAR	Aquatic, skilled traders
Vesh	TAVON	Hive mind, subsonic communicators
The Remnant	VARYN	Ancient AI collective
Wanderers	Fleet-born	Nomadic traders, expert shiphandlers

Anomalies

Some systems contain unexplained phenomena with potential strategic implications:

Anomaly	System	Description
Whisper Gate	BELIX	Ancient structure; energy readings persist
Core Fragment	CAYRU	Exotic matter in The Shatter belt

First Library	ZAREK	Elder Race data storage
Ishtar Gate	ASTREX	Metal ring, 20m across, ancient texts mention it
Axis Marker	KORAL	Navigation beacon, warpline efficiency
Keplerite Vein	QUELL	Warp-enhancing exotic crystals
The Graveyard	SYDRA	Powered wreckage field, high salvage value
The Song	TAVON	Vesh vibrations that penetrate vacuum

Notable Resource Deposits

Resource	Location	Abundance
FERROUS	The Anvil (ARVEN)	Rich
FERROUS	The Shatter (CAYRU)	Rich
FERROUS	Servitor (ZAREK)	Rich
RARE_EARTH	Jewel (BELIX)	Rich
RARE_EARTH	The Shatter (CAYRU)	Rich
VOLATILE	Shepherd (ARVEN)	Rich
VOLATILE	Cloud Marshal (BELIX)	Rich
ORGANIC	Verdance (ARVEN)	Rich
ORGANIC	Hanging Gardens (ASTREX)	Rich
EXOTIC	Keplerite (QUELL)	Rich
EXOTIC	Gilded Throne ruins (ZAREK)	Rich
WATER	Verdance (ARVEN)	Abundant
WATER	Hanging Gardens (ASTREX)	Abundant
CRYSTALLINE	Glassine (XYLEN)	Rich

AI AGENT

Kepler's Horizon supports single-player mode through an autonomous AI opponent called the **Autonomy Agency**. The AI replaces one human player and executes commands through the same game engine, respecting all rules and constraints. It issues the same commands a human player would type.

The AI thread runs independently of HTTP request handling. When a game ends (via **quit**, victory, or elimination), the AI's game context is zeroed—it is **never killed**. The thread stays alive and naturally ceases operating when it detects **m_game_id = 0** on its next gather cycle.

Architecture: Mealy State Machine

The AI uses a three-phase decision cycle:

1. GATHER (C++)

Populate an **AASlate** data structure from the database and StateMachine. The **Slate** is the *Slate of State* which means a scratch board of *metrics* that are gleaned from sources within the game, during steps conducted by the State Machine of the AI-Agency. The slate contains the complete observable game: fleet positions, combat theaters, economic data, territory control, BFS distance matrix, hex adjacency, warpline topology, codex entries, market prices, and cross-turn metrics.

2. CALCULATE (Lisp)

Marshal the slate to Common Lisp via ECL (Embeddable Common Lisp), call the strategy function (**aa-calculate slate**), and unmarshal the result: a list of command strings and optional metric updates.

3. RENDER (C++)

Inject the resulting command string into the game engine's command queue via **AICommandInjector**, exactly as if a human typed it. Block until the TaskRunner completes the command. The game engine is designed to handle two-player or single-player modes. ALL commands are funneled through **yyparse()** -- even the AI-Agency commands -- this is mostly to avoid duplicative work in the server-side and also make use of the return-codes (success or failure) that may be obtained by the commands themselves. **TaskRunner** as a thread simply invokes commands regardless of their initiator. User-player or AI-Agency. But, the AI-Agency, when enabled in single-player mode pays attention to the results of the Task Runner and always modifies the Slate and therefore may modulate the *next command* based on outcomes from the prior AI-Agency invoked commands. The AI-Agency is adaptive.

The cycle repeats (gather fresh state, recalculate, render) until the AI issues a terminal command (**NEXT** or

DONE) to advance the turn. This closed-loop design means every decision is based on current game state, not stale assumptions.

A safety guard at the top of **gather()** checks **m_game_id** ≤ 0 and sets **game_over** = **true**, causing the Mealy loop to exit gracefully without querying the database.

Strategy DSL Modules

The AI strategy is implemented across eight Common Lisp modules, loaded in dependency order:

aa-util.lisp

Accessor functions for reading the data slate. Ship property lookups, list filters, and helper predicates.

aa-strategy.lisp

High-level strategic ontology (A1–A11). Computes posture (aggressive/defensive/balanced), theater coordination, and inter-phase directives. Uses plist flow: **compute-posture** → theater coordination → build/movement directives.

aa-goals.lisp

Goal evaluation engine using model-predictive control. Evaluates objectives in priority order and finds the highest-priority unsatisfied goal.

aa-economic.lisp

Maps economic goals to concrete commands: survey, extract, buy, fabricate, sell, outfit, salvage.

aa-build.lisp

Ship design selection, construction, repair, and deployment decisions. Selects from predefined ship templates based on game state.

aa-movement.lisp

Movement routing with BFS pathfinding. Prioritizes defending threatened bases and pushing ships toward undefended enemy bases for VP capture. Spreads ships across different enemy bases to maximize VP coverage.

aa-combat.lisp

Multi-theater triage, CRT-aware power allocation, target selection (focus fire on lowest-HP enemy), damage assignment (sacrifice tubes first, then beams, screens, PD last).

aa-core.lisp

Main entry point and phase dispatcher. Routes decisions by turn phase (build, movement, combat, pick/drop, end turn).

Build Strategy

The AI selects from predefined ship templates based on game state:

Template	Attributes	Role
Brawler	PD=6 B=4 S=3	Main battle ship; built first
Interceptor	PD=5 B=2 S=2	Fast pursuit; spreads to bases
Missile Boat	PD=4 S=1 T=2 M=6	Alpha strike; counters heavy screens
Fortress	PD=8 B=6 S=5	SystemShip for base defense

Template selection considers enemy fleet composition, tech level timing, and fleet diversity. The AI caps its fleet at 5 WarpShips.

Combat Strategy

The AI assesses all active combat theaters simultaneously:

- Force ratio (our PD / their PD) determines fight/hold/retreat
- VP hexes (enemy bases) are defended more aggressively
- All ships in one hex focus fire on the same target (lowest total HP)
- Drive differential is clamped to avoid +5 automatic miss

- Damage assignment: sacrifice tubes first, then beams, screens, PD last

Economic Strategy

The AI evaluates economic objectives in priority order:

1. Resupply missiles at base (if stock below 50%)
2. Fabricate missiles (if materials available at shipyard)
3. Acquire resources needed for fabrication (extract or buy)
4. Survey unknown systems for resource yields
5. Outfit ships with drones (enables salvage/extraction)
6. Salvage wreckage at anomaly sites
7. Sell excess cargo at trade hubs

Cargo is never sold while it serves an active goal.

Cross-Turn Memory

The AI persists deduced metrics to the **aa_metrics** database table via **(make-metric name value)** pseudo-commands emitted by Lisp. These metrics survive across decision cycles and are loaded back into the slate each gather.

SYSTEMSHIP PICKUP AND DROP

During Movement Phase

- Costs 1 PD per SystemShip picked up or dropped
- Can be performed at any star hex during movement

During Combat

- Requires: Drive=0, Screen=0, Dodge or Escape tactic
- Only 1 SystemShip per combat round
- May fire Beam (but not Missiles) while picking up/dropping
- Dropped SystemShips cannot fire that round
- Picked up SystemShips cannot fire but may power Screens

After Combat (Phase 5)

- Free rearrangement of SystemShips at star hexes
- No PD cost
- Active player only

BUILD POINT COSTS

Attribute	Cost
Power/Drive (PD)	1 BP per unit
Warp Generator (WG)	5 BP (WarpShips only, automatic)
Beam (B)	1 BP per unit
Screen (S)	1 BP per unit
Tube (T)	1 BP per tube
Missiles (M)	1 BP per 3 missiles
SystemShip Rack (SR)	1 BP per rack (WarpShips only)

CONSTRAINTS AND INVARIANTS

These are the fundamental laws of the game engine. They are never violated, and no command or sequence of commands can circumvent them:

- No randomness in combat—all outcomes are deterministic.
- No hidden information—all game state is public.

- Rules do not change mid-game.
- Equal inputs yield equal outcomes.
- Ships retain original tech level permanently.
- Warp Generators never take damage. A ship is destroyed when all attributes except WG reach 0.
- SystemShips cannot have Warp Generators or SystemShip Racks.
- WarpShips cannot be carried in SystemShip Racks.
- Beams/Screens and Missiles cannot be used in the same combat round.
- SystemShips may never select the Escape tactic.
- PD used for movement is not permanently expended.
- Repairs cannot exceed original built capacity.
- Missile resupply cannot exceed original missile capacity.

EXAMPLES

Building a WarpShip

- > **bn** W Falcon
- > **bs** Falcon pd=5 b=3 s=2 t=1 m=3
- > **bc** Falcon

Total cost: PD(5) + B(3) + S(2) + T(1) + M(1) + WG(5) = 17 BP. Tech level determined by current round.

Building a SystemShip

- > **bn** S Avenger
- > **bs** Avenger pd=1 t=1 m=6
- > **bc** Avenger

Total cost: PD(1) + T(1) + M(2) = 4 BP. No Warp Generator required.

Movement

- > **m** W6 Tavon Xylen

W6 moves from current hex to Tavon via the nearest warpline path, then onward to Xylen. Total PD cost depends on path length (BFS-computed).

Combat Sequence

- > **co** W3 a S25 pd=2 b=3 s=2
- > **cc**
- (Wait for opponent to commit...)
- (Resolution: W3 hits S25 for 3+tech damage)
- > **ca** S25 pd=2 b=1

Missile Fire

- > **co** S25 d W3 pd=4 t=1 m=3
- > **cc**

S25 dodges while firing 1 missile at W3. The missile uses Attack tactic with drive=3. Missile damage on hit = 2 + tech level.

Economic Workflow

- > **out** W1 lrs
- > **survey** ARVEN
- > **extract scan**
- > **ex** W1 ferrous
- > **market**
- > **trade sell** ferrous 5
- > **fab** missiles W1

Save and Load

- > **save** my-game
- > **load** my-game
- (Other player must type:)
- > **accept** my-game

Quit

- > **quit**
- QUIT: Game auto-saved as 'autosaved-2026-02-13-14-30'.
- Returning to lobby.
- The game is auto-saved. Combat state is cleared. The other player is notified and redirected. The AI thread ceases operating on this game.

MODULES**Creating Custom Modules**

A Module is a complete universe definition loaded from CSV files. All milieu data (star systems, planets, species, resources) is keyed by **module_id**.

Required files:

File	Content
star_systems.csv	Map topology (hex_id, name, base flags)
hexes.csv	Hex coordinates (q, r)
warplines.csv	Warpline connections (a_hex, b_hex)
warpline_hexes.csv	Intermediate warpline path hexes
planets.csv	Planetary bodies
species.csv	Xenofom species definitions
resources.csv	Extractable resource deposits
facilities.csv	Facilities and stations
anomalies.csv	System anomalies
salvageables.csv	Salvage sites and hazards
codex_rumors.csv	Knowledge and rumors per system

See the **ModuleAuthoring** wiki page for complete authoring instructions.

HEX EVENTS

Dynamic events affect specific hexes, modifying gameplay:

Event Type	Effect
NAVIGATION_HAZARD	+N PD cost to move through hex
COMBAT_INTERFERENCE	-N beam damage in hex
SALVAGE_OPPORTUNITY	+25% salvage yields
EXTRACTION_BONUS	+N extraction yield

Events spawn and expire based on turn count. Use **hex location** to view active events.

MONITOR MODE

The server may accept commands from the terminal when started with **--monitor**. Each command must be prefixed with a player indicator. This feature is intended for development testing.

DATABASE

The game uses MySQL/MariaDB. The schema is defined in **site/db/Game.sql**. Module data is loaded from CSV files via **LoadModule.sql** scripts. Core data (help topics) is loaded via **site/db/core/Load.sql**.

Key tables: **games**, **ships**, **drafts**, **combat_state**, **combat_orders**, **pending_damage**, **star_systems**, **warplines**, **system_resources**, **codex_entries**, **market_prices**, **facility_control**, **hex_events**, **aa_metrics**, **saved_games**.

Machine Learning: AI Agent Feedback Loop & Latent Learning

Design issues about closing the feedback loop between command execution and AI agent decision-making, and a longer-term vision for build-time machine learning driven by game diaries.

1. The Problem: Open-Loop Command Execution

The AI agent's MPC (model-predictive control) loop is currently open-loop at the RENDER step. The cycle is:

```
GATHER -> CALCULATE -> RENDER -> (assume success) -> GATHER
```

A human player closes this loop naturally -- they see "Invalid move" on screen and adjust. The AI agent has no equivalent feedback channel.

When `invoke()` returns false on a command, the TaskRunner knows the command failed and logs the error. But `notify_task_complete()` is a bare signal -- it carries no payload. The agent wakes up from `wait_for_task_completion()` knowing only "the task finished," not whether it succeeded or failed, or why.

This causes blind retry loops: the agent re-issues the same failing command because it never learns it failed. The game state hasn't changed, so CALCULATE derives the same command from the same inputs.

2. The Gap Is Narrow

The information already exists at the right place. In `taskrunner.cpp`, after executing an AI task:

```
int result = internal_command_handler_body(item->ai_command, errmsg);
```

The TaskRunner has both `result` (success/failure) and `errmsg` (the reason). It logs them, then calls `notify_task_complete()` which discards both. The plumbing for the blocking handshake between TaskRunner and AI agent thread already exists via condition variable -- it just signals a bool when it could signal a result structure.

3. Proposed Feedback Mechanism

Task Meta-Data

Each Task carries meta-data that is populated by the grammar rule involved in the `invoke()` call. This is not just a bool -- it includes contextual information about *why* the command failed:

- The command that was attempted
- Success or failure
- Structured failure reason (not a human-readable string)

Notification With Payload

When `notify_task_complete()` fires, the meta-data blob is conveyed to the AI agent. The agent only cares about failures -- successful commands need no special handling since the world state changes and the next GATHER reflects reality.

Persistence in the Slate

Failed-command meta-data persists in the agent's awareness and becomes part of the slate during the next GATHER step. The Lisp code can then see: "my last MOVE w6 2223 failed because X" and choose differently.

4. Failure Reasons as Ontology Facts

Rather than carrying free-form error strings that Lisp would have to parse, failure reasons should be closed under the ontology dictionary. Every failure cause maps 1:1 to an ontology fact:

- "insufficient credits" -> `:precondition-violated :credits-below-threshold`
- "hex contested" -> `:precondition-violated :hex-in-combat`
- "not your turn" -> `:precondition-violated :phase-mismatch`

This means the Lisp code reasons about failures the same way it reasons about everything else -- through the strategy plist, using the same subsystem functions. Failures become just another structured input to CALCULATE, not a special side channel.

The closed-set property guarantees the Lisp code can handle every failure because the ontology is finite and known at compile time.

5. Failure Lifetime: When to Forget

Time-Based Expiry Is Wrong

A fixed TTL (e.g., "forget after 3 rounds") is either too long -- the agent avoids a now-valid command for rounds after the blocker cleared -- or too short -- the agent retries a still-invalid command because the timer expired.

State-Change Invalidation Is Ideal But Hard

The theoretically correct approach: a failure stays relevant until the slate shows the precondition that caused it has changed. But this requires mapping each failure reason to the specific state that caused it, which is a complex, fragile mapping.

Phase-Scoped Lifetime Is Pragmatic

Clear the failure list at each phase transition. Within a single phase, game state only changes through the agent's own actions (and the other player's during combat). A command that fails in BUILD phase round 5 won't succeed later in BUILD phase round 5 -- the preconditions are stable within a phase.

Exception: combat, where state changes within a phase as orders resolve, damage is assigned, and ships escape. Combat may need round-scoped or stage-scoped clearing.

6. Latent Machine Learning: The Diary Concept

Two Timescales

Separate the learning feedback loop into two timescales:

- **Runtime (within a game):** Static Lisp, fixed ontology, metrics memory. No self-modification. Deterministic and debuggable.
- **Build-time (between releases):** Ingest accumulated game diaries, produce updated Lisp parameters. The agent evolves, but only through a controlled, auditable build step.

The Diary

Extend the existing `aa_metrics` persistence to capture richer game-over summaries: what round the game ended, who won, fleet composition at key moments, which strategies were attempted and when. Structured data, not prose.

Translation Approaches (Build-Time Ingestion)

Three approaches, ordered by complexity:

Approach 1: Parameter Tables (Recommended Starting Point)

The diary feeds a statistical pass that produces a config file -- thresholds, weights, timing windows. Example: "average earliest viable missile build = round 5.2 across 47 games" becomes a `defparameter` value. No new Lisp code is generated. The build step is data aggregation. Lisp code has `defparameter` declarations that get values from a generated config.

Simplest, safest, and likely captures 80% of the benefit.

Approach 2: Rule Selection

Write multiple strategy variants by hand -- aggressive early-build, conservative turtle, missile-rush, etc. Diary analysis determines which variants performed best against different opponent profiles. The build step produces a selection table. No generated Lisp, just generated configuration selecting among pre-written variants.

Approach 3: Lisp Code Generation

An external tool reads the diary, identifies patterns, and emits new Lisp functions or modifies existing ones. Requires a validation step -- generated Lisp must type-check against the ontology, return well-formed plists, and not introduce infinite loops. This is a research project more than an engineering task.

Build System Integration

For approaches 1 and 2: a pre-build script reads the diary database, runs the analysis, writes a `.lisp` file of `defparameter` declarations or a strategy-selection table, and the normal build includes that file. No special tooling beyond a script that queries the metrics database and emits s-expressions.

On Self-Modifying Lisp

The power of the current architecture is that Lisp functions are pure: slate in, decisions out. They carry no hidden state, don't mutate themselves, don't evolve at runtime. That makes them debuggable, predictable, and auditable. Self-modifying code at runtime sacrifices all three properties. The diary/build-time approach preserves them by restricting evolution to a controlled, versioned build step.

BUGS

Report bugs to the Kepler's Horizon issue tracker.

FILES

Files that are deployed when installing the system:

site/web/

Static HTML, CSS, JavaScript files served by the HTTP layer. These are deposited in the directory where your web server is accessing content.

site/db/Game.sql

Complete database schema. Stored securely, used to initiate the database back-end.

site/db/core/help/

Help topic CSV files for the in-game help system. Stored securely, used by the database initialization setup.

site/db/modules/kh/

Module data CSV files for the default Kepler's Horizon universe. Stored securely, used by the database initialization setup.

dsl/ AI strategy DSL files (Common Lisp). A directory containing the run-time *LISP* code.

AUTHORS

Game design and implementation by sibomots.

ACKNOWLEDGEMENT

Game design is based upon the table-top (pencil and paper) game called **WarpWar** that was released in 1977. <https://en.wikipedia.org/wiki/WarpWar> Kepler's Horizon has extended the game much further, adding milieu concepts for market (trading, selling resources), resource extraction, exploration, fabrication, random space hazards, and an AI-Agent for single player mode.

FORMAT

This document is set in the style of traditional *UNIX* manual pages. It was prepared using *troff* (*equivalent*) with the **man**(7) macro package.

```
groff -t -man -Tpdf kh.6 > kh.pdf
```

Sectioning, typography, and layout follow established UNIX documentation conventions. No additional formatting mechanisms are employed.

COPYRIGHT

Copyright (c) 2025 sibomots. Licensed under BSD 3-Clause License.